

Sets – creation, adding and removing elements.

सेट्स (Sets) – सेट बनाना, तत्व जोड़ना और तत्व हटाना।

Python में सेट (Set) **अद्वितीय तत्वों (unique elements)** का एक **अक्रमबद्ध (unordered)** संग्रह है। सेट्स **परिवर्तनशील (mutable)** होते हैं, डुप्लिकेट मानों (duplicate values) की अनुमति नहीं देते और गणितीय सेट ऑपरेशंस (Mathematical Set Operations) का समर्थन करते हैं।

1. Sets – Creation | 1. सेट बनाना

हम कर्ली ब्रेसिस {} या set() कंस्ट्रक्टर का उपयोग करके सेट बना सकते हैं:

```
a = {1, 2, 3, 4}           # using curly braces
b = set([1, 2, 3, 4])      # using set() constructor
c = set('hello')           # duplicates removed
d = set()                  # empty set
```

OUTPUT | आउटपुट

```
a → {1, 2, 3, 4}
b → {1, 2, 3, 4}
c → {'h', 'e', 'l', 'o'}
d → set()
```

2. Adding Elements to a Set | 2. सेट में तत्व जोड़ना

हम add() या update() मेथड का उपयोग करके सेट में नए तत्व जोड़ सकते हैं:

```
s = {10, 20}
s.add(30)           # adds single element
s.add(20)           # 20 exists, no change
s.update([40, 50])  # adds multiple elements
```

OUTPUT | आउटपुट

```
s → {10, 20, 30, 40, 50}
```

3. Removing Elements from a Set | 3. सेट से तत्व हटाना

हम विभिन्न मेथड्स का उपयोग करके सेट से तत्व हटा सकते हैं:

```
s = {10, 20, 30, 40, 50}
s.remove(20)         # KeyError if not found
s.discard(100)       # no error if not found
popped = s.pop()     # removes random element
s.clear()             # removes all elements
```

OUTPUT / RESULT | आउटपुट

```
After remove(20) → {10, 30, 40, 50}
After discard(100) → {10, 30, 40, 50}
popped → (random element)
After clear() → set()
```

Key Points मुख्य बिंदु

- सेट्स केवल **अद्वितीय तत्वों (unique elements)** को स्टोर करते हैं (डुप्लिकेट्स अपने आप हट जाते हैं)।
- सेट्स में तत्वों का **क्रम (order)** सुरक्षित नहीं रहता है।
- तत्व जोड़ने के लिए add() या update() का उपयोग करें।
- तत्व हटाने के लिए remove(), discard(), pop() या clear() का उपयोग करें।

© Vinay Soni

Sets – creation

सेट्स (Sets) – सेट बनाना (Creation)

Python में सेट (Set) **अद्वितीय तत्वों (unique elements)** का एक **अक्रमबद्ध (unordered)** संग्रह है।
सेट्स **परिवर्तनशील (mutable)** होते हैं, डुप्लिकेट मानों की अनुमति नहीं देते और गणितीय सेट ऑपरेशंस का समर्थन करते हैं।

1. Creating Sets | 1. सेट बनाना

हम कर्ली ब्रेसिस {} या set() कंस्ट्रक्टर का उपयोग करके सेट बना सकते हैं:

```
a = {1, 2, 3, 4}           # using curly braces
b = set([1, 2, 3, 4])      # using set() constructor
c = set('hello')           # duplicates removed
d = set()                  # empty set
```

OUTPUT | आउटपुट

```
a → {1, 2, 3, 4}
b → {1, 2, 3, 4}
c → {'h', 'e', 'l', 'o'}
d → set()
```

Explanation | विस्तृत स्पष्टीकरण

कोड (Code)	प्रक्रिया (Process)	विस्तृत विवरण (Detailed Explanation)
a = {1, 2, 3, 4}	→	कर्ली ब्रेसिस {} का उपयोग करके सेट बनाया गया।
b = set([1, 2, 3, 4])	→	लिस्ट को set() कंस्ट्रक्टर में पास करके सेट बनाया गया।
c = set('hello')	→	स्ट्रिंग के अक्षर अद्वितीय तत्वों के रूप में जुड़े, डुप्लिकेट 'l' हट गया।
d = set()	→	एक खाली सेट (empty set) बनाया गया।

विधि (Method)	सिंटैक्स (Syntax)	उदाहरण (Example)	प्रकार (Type)
Curly Braces	{item1, item2}	{10, 20, 30}	<class 'set'>
set() Constructor	set(iterable)	set([1, 2, 2, 3]) → {1, 2, 3}	<class 'set'>
Empty Set	set()	empty_set = set()	<class 'set'>

Key Points मुख्य बिंदु

- सेट्स **अक्रमबद्ध (unordered)** होते हैं और इनमें इंडेक्सिंग s[0] का समर्थन नहीं होता है।
- डुप्लिकेट मान (Duplicate values) स्वतः हटा दिए जाते हैं।
- सेट के अंदर मौजूद तत्व हमेशा **अपरिवर्तनीय (immutable)** (जैसे numbers, strings, tuples) होने चाहिए।
- खाली सेट बनाने के लिए set() का उपयोग करें (क्योंकि {} खाली डिक्शनरी बनाता है)।

Note / ध्यान दें : Python में {} लिखने से **Dictionary** बनती है, Set नहीं। खाली सेट बनाने के लिए हमेशा set() का ही उपयोग करें।

© Vinay Soni

Sets – adding

सेट्स (Sets) – तत्व जोड़ना (adding elements)

हम `add()` या `update()` मेथड का उपयोग करके सेट में तत्व जोड़ सकते हैं।

चूँकि सेट्स अक्रमबद्ध (unordered) होते हैं और डुप्लिकेट की अनुमति नहीं देते, इसलिए केवल **अद्वितीय तत्व (unique elements)** ही जोड़े जाते हैं।

1. Using add() Method | 1. add() मेथड का उपयोग

`add()` मेथड सेट में एक बार में **एकल तत्व (single element)** जोड़ता है:

```
s = {10, 20, 30}          # initial set
s.add(40)                  # adds 40
s.add(20)                  # 20 already exists, no change
s.add('hello')             # adds 'hello'
print(s)                   # output the set
```

OUTPUT | आउटपुट

```
{10, 20, 30, 40, 'hello'}
```

2. Adding Multiple Elements | 2. कई तत्व जोड़ना (update Method)

हम `update()` मेथड द्वारा किसी भी इटरेबल (list, tuple, set, string) से एक साथ कई तत्व जोड़ सकते हैं:

```
s = {1, 2}                 # initial set
s.update([3, 4, 5])         # add elements from list
s.update({5, 6, 7})         # add elements from set
s.update('ab')              # add characters from string
print(s)
```

OUTPUT | आउटपुट

```
{1, 2, 3, 4, 5, 6, 7, 'a', 'b'}
```

मेथड (Method)	आर्गुमेंट प्रकार (Argument Type)	क्या करता है (Action)	डुप्लिकेट व्यवहार (Duplicate Handling)
<code>s.add(item)</code>	एकल मान (Single value)	सेट में 1 तत्व जोड़ता है	यदि पहले से है, तो कोई बदलाव नहीं
<code>s.update(iterable)</code>	इटरेबल (List, Tuple, Set, Str)	सभी अद्वितीय तत्वों को जोड़ता है	डुप्लिकेट्स स्वतः फ़िल्टर हो जाते हैं

Key Points मुख्य बिंदु

- `add()` सेट में केवल एक एकल तत्व जोड़ता है।
- यदि तत्व पहले से मौजूद है, तो सेट अपरिवर्तित रहता है (कोई एरर नहीं आता)।
- `update()` किसी भी इटरेबल (list, set, tuple, string आदि) से एक साथ कई तत्व जोड़ता है।
- सेट में केवल अद्वितीय तत्व (unique elements) ही संग्रहीत होते हैं।

Best Practice / सुझाव : जब एक मान जोड़ना हो तो `add()` का उपयोग करें, और जब किसी पूरी लिस्ट या सीक्वेंस के तत्वों को सेट में मिलाना हो तो `update()` का उपयोग करें।

© Vinay Soni

Sets – removing elements.

सेट्स (Sets) – तत्व हटाना (removing elements)

हम विभिन्न मेथड्स का उपयोग करके सेट से तत्व हटा सकते हैं।
यदि तत्व सेट में नहीं मिलता है, तो कुछ मेथड्स **KeyError** एरर उत्पन्न करते हैं जबकि कुछ बिना एरर के शांत रहते हैं।

1. Using remove() Method | 1. remove() मेथड द्वारा हटाना

विशिष्ट तत्व को हटाता है। यदि तत्व मौजूद नहीं है, तो यह **KeyError** एरर फेंकता है:

```
s = {10, 20, 30, 40, 50} # initial set
s.remove(30)             # removes 30
s.remove(60)             # KeyError (60 not found)
print(s)                 # output the set
```

OUTPUT / RESULT | आउटपुट
After remove(30) → {10, 20, 40, 50}
After remove(60) → **KeyError**

2. Using discard() Method | 2. discard() मेथड द्वारा हटाना

विशिष्ट तत्व को हटाता है। यदि तत्व मौजूद नहीं है, तो कोई एरर उत्पन्न नहीं होता:

```
s = {10, 20, 30, 40, 50} # initial set
s.discard(20)             # removes 20
s.discard(60)             # no error (60 not found)
print(s)                 # output the set
```

OUTPUT | आउटपुट
After discard(20) → {10, 30, 40, 50}
After discard(60) → {10, 30, 40, 50}
(no change)

3. Using pop() Method | 3. pop() मेथड द्वारा हटाना

सेट से एक यादृच्छिक (random) तत्व हटाता है और लौटाता है। खाली सेट पर यह **KeyError** देता है:

```
s = {10, 20, 30}         # initial set
x = s.pop()              # removes random element
print("Removed:", x)     # print removed element
print("Remaining set:", s)
```

OUTPUT / RESULT | आउटपुट
Removed: 20 (random element)
Remaining set: {10, 30}

4. Using clear() Method | 4. clear() मेथड द्वारा हटाना

सेट के सभी तत्वों को हटाकर उसे पूरी तरह खाली कर देता है:

```
s = {1, 2, 3, 4}         # initial set
s.clear()                 # removes all elements
print(s)                 # output the set
```

OUTPUT | आउटपुट
set() (empty set)

मेथड (Method)	क्या हटाता है (Deletes)	रिटर्न वैल्यू (Return)	यदि तत्व न मिले (If Item Missing)
s.remove(x)	विशिष्ट तत्व x	कुछ नहीं	KeyError एरर फेंकता है
s.discard(x)	विशिष्ट तत्व x	कुछ नहीं	कोई एरर नहीं (सुरक्षित)
s.pop()	यादृच्छिक (Random) तत्व	हटाया गया तत्व	KeyError (यदि सेट खाली हो)
s.clear()	सभी तत्व	कुछ नहीं (सेट set())	कोई एरर नहीं

Key Points मुख्य बिंदु

- remove(x) → तत्व x को हटाता है, न मिलने पर **KeyError** देता है।
- discard(x) → तत्व x को हटाता है, न मिलने पर एरर नहीं देता।
- pop() → एक यादृच्छिक (random) तत्व हटाता है और लौटाता है।
- clear() → सेट के सभी तत्वों को हटा देता है।

© Vinay Soni